

A Task-Specific Visual Token Access Threshold in Vision-Language Models

Implications for token pruning

P. S. Kedar

Supervised by Prof. Divya Saxena | Mentored by Aditya Sharma, PhD Scholar | IIT Jodhpur, September 2026

1. Objective and observation

Token pruning throws away visual tokens while a model runs, to save compute. Balanced Token Pruning does it in three stages. We reproduced its published numbers in earlier phases and they held up, except on one thing: Qwen2.5-VL-7B lost almost all its ability to read text inside an image. TextVQA fell from 86.2 to 23.3. DocVQA fell from 94.7 to 19.2. AI2D, which has text but rarely needs it read word by word, barely moved.

Phase 3 set out to explain that gap. Four experiments were run to distinguish between candidate causes. All four returned negative, and the results eventually moved the investigation from token selection to the handling of visual tokens after pruning.

Summary of findings

On TextVQA there is a depth beyond which deleting every visual token costs the model very little, and before which the loss is severe. The transition is abrupt. We measure it at 74.8 per cent of decoder depth in InternVL2-2B and 80.3 per cent in Qwen2.5-VL-7B. The threshold is task-specific: it is measured on one task and may sit elsewhere for others.

The released BTP code deletes at a point that leaves Qwen 22 layers with image access, against a measured threshold of 22.5. Near that threshold one layer accounts for about 47.7 percentage points on TextVQA.

The deletion is not the only cost. On ChartQA the graded pruning costs roughly twice what the deletion does, so the two are reported separately throughout.

Neither cost is visible on the benchmarks the paper reports. With the discrepancy present the code still retains 96.1 per cent of baseline across five of them.

We measured the thresholds on TextVQA only, on two models, and our InternVL pruning is not a complete reproduction of BTP. Section 6 states where that matters.

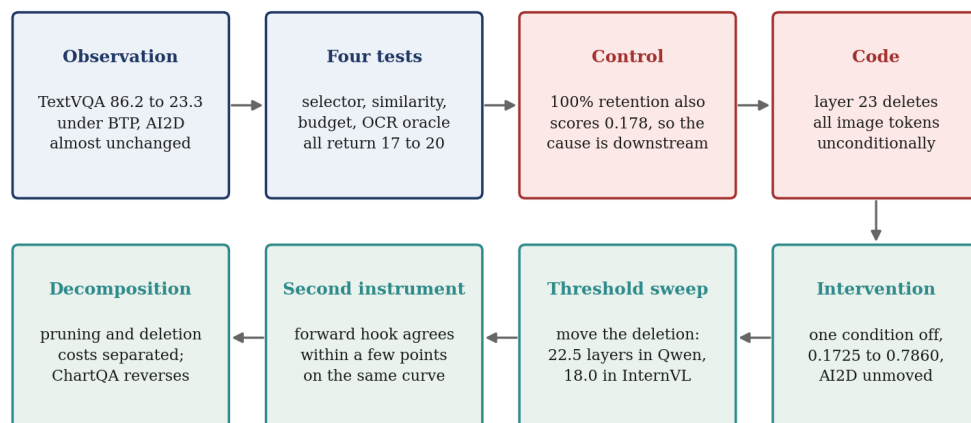


Figure 1. The investigation, in order. The upper row eliminated candidate causes; the control at 100 per cent retention redirected it downstream. The lower row is the work that followed from the code finding.

2. Four alternative explanations tested

We enumerated the plausible causes and designed a test capable of eliminating each.

Initial hypothesis	Experimental test	Result and interpretation
The selector picks badly	Swapped it for diversity only, attention only, and random, at the same budget	17.3 to 19.4, standard errors 2.6 to 2.7. Random did as well as BTP
Text patches look alike, so a diversity sampler thins them	Measured feature similarity at layer 4	Text-text 0.379, background-background 0.438. Text is the more varied region, so our premise was wrong
Too many tokens are removed	Swept retention from 100 down to 12.5 per cent	Keeping 90 per cent already gave the full drop, 86.2 to 23.3. Keeping 12.5 gave 23.3
The right tokens are not being kept	Used ground-truth OCR to force every text token to survive	0.197 with the oracle, 0.197 without

Table 1. Four negatives, in order. All four landed on the same floor.

Four unrelated interventions produced the same number, which was itself the signal. The point became clear only after we added a control at 100 per cent retention, where nothing is pruned at all. It scored 0.178. Retaining every token changed nothing, so the loss was occurring after the pruning stages rather than during them.

3. The implementation discrepancy and its causal test

We inspected the released Qwen file for operations that alter the hidden-state sequence. One branch is responsible:

```
elif layer_index == self.end_layer:      # self.end_layer = 23
    remain_sys_index = torch.tensor(list(range(self.img_start_idx)))
    remain_text_index = torch.tensor(list(range(img_end, total_length)))

    all_remained_index = torch.cat((remain_sys_index, remain_text_index))
    hidden_states = hidden_states[:, all_remained_index, :]
```

The concatenation includes system tokens and text tokens. It does not include image tokens. There is no `remain_img_index`, so every token surviving the three pruning stages is removed at this point regardless. A commented-out variant immediately above would have retained a subset by attention score.

Code inspection alone is not evidence of cause. We disabled the single condition so the branch cannot fire, left the 12.5 per cent pruning unchanged, and re-ran the evaluation.

Arm	TextVQA	AI2D
No pruning	0.8565	0.8750
BTP as released	0.1725	0.8150
Deletion disabled	0.7860	0.8100

Table 2. Job 416735, 200 samples. The 0.8565 baseline differs from the 86.2 used elsewhere because this ran on a separate 200 sample subset. Code: `NeurIPS2025-Balanced-Token-Pruning` at commit 9682db0, file `qwen-2.5-vl/modeling_qwen2_5_vl.py`, constants at lines 1140 to 1144, branch at line 1385, read unmodified.

Changing one condition moves reading accuracy from 0.1725 to 0.7860. AI2D is unchanged, which matters: it shows the intervention is not simply a lever that raises every score.

The paper is worth checking here. Appendix 7.3 specifies complete deletion for the LLaVA family. For Qwen2.5-VL it asks for the opposite: keep 12.5 per cent in the final stage, in the authors' words, to preserve model performance. The released code we examined applies the LLaVA branch to Qwen regardless. We describe this as a discrepancy in the released implementation. It says nothing about what the authors intended, and nothing at all about Qwen2.5-VL as a model.

4. Measuring a task-specific visual token access threshold

A poorly chosen constant is less informative than the quantity that makes it poor. We therefore moved the deletion across the whole decoder with graded pruning disabled, changing nothing else, and repeated the procedure on InternVL2-2B. That model shares almost nothing with Qwen: a 24 layer InternLM2 backbone, ordinary 1D RoPE instead of M-RoPE, 448 pixel tiling instead of a merged patch grid.

A note on counting

We report the number of layers that still had the image, counting from the first: deleting before layer 12 of 28 means twelve had it and sixteen did not. That is not the number a method writes in its code. BTP counts from one and deletes before the named layer, so its end_layer of 23 leaves twenty-two layers with access. We count from zero. An earlier draft of this report conflated the two.

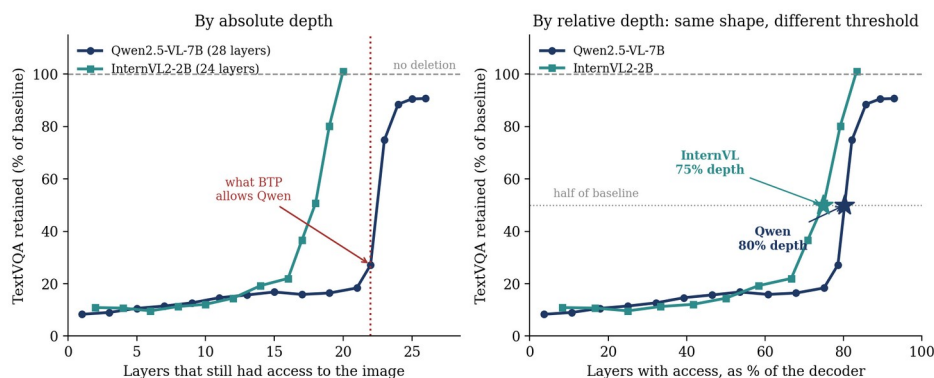


Figure 2. Left, by layers with access. Right, by relative depth. Both models sit on a floor, then climb sharply.

We take the threshold to be where accuracy crosses half the unpruned baseline, interpolated between the two measured layers either side. The baselines for these sweeps are 0.8624 for Qwen and 0.7156 for InternVL, both TextVQA at limit 500. A different cutoff, say 90 per cent, moves both numbers, so treat these as a comparison between the models rather than absolutes.

Model	Decoder layers	Threshold	Relative depth
InternVL2-2B	24	18.0	74.8 %
Qwen2.5-VL-7B	28	22.5	80.3 %

Table 3. Layers that still need the image.

Two observations follow. The curve has the same shape in both models. With two models that is a pattern worth testing further rather than a property of vision-language models in general. The depths differ, so the value cannot be inferred from the layer count alone.

BTP's constant leaves Qwen 22 layers. We measure 22.5. At 22 layers the model keeps 27.2 per cent of baseline; at 23 it keeps 74.8. The released value therefore sits just below the threshold, and one layer accounts for roughly 47.7 percentage points.

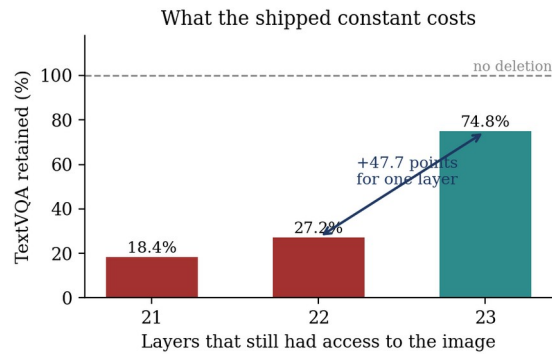


Figure 3. Either side of the threshold. The 47.7 comes from the unrounded scores; the rounded percentages shown differ by 47.6.

One qualification about what is being measured. We observe the effect of removing the tokens, not the model's use of them, so this is a statement about sensitivity rather than mechanism. Describing it as the point where the model stops looking at the image would claim more than the curve supports.

5. Validating the measurement with an independent instrument

Both sweeps edit model source, separately for each architecture, which limits reuse. We therefore rebuilt the measurement as a forward hook that removes the tokens during the forward pass without altering any source, and ran it on Qwen, where the answer was already known.

Layers with access	Source patch	Forward hook
13	15.7 %	17.0 %
22	27.2 %	27.5 %
23	74.8 %	75.8 %
24	88.5 %	86.3 %

Table 4. Job 423959, different sample sets, so compare the threshold and not the absolute scores.

The two agree in shape and to within a few points. This supports the measurement itself rather than any particular way of making it, though the comparison has been run on one model only. The tool needs no training and no calibration set, and a coarse sweep plus bisection finds the threshold in about seven runs.

The two models differ at the top of the curve, which we had not anticipated. InternVL recovers fully, reaching 101 per cent of baseline with 20 of its 24 layers. Qwen plateaus at 90.7 per cent even with 26 of 28, which suggests visual information remains useful very late in its decoder for some subset of examples. This experiment does not identify that subset or its size.

6. Separating pruning cost from deletion cost

With the mechanism understood we ran a corrected arm across the benchmarks. It still prunes to 12.5 per cent; it just does not delete what is left. That gives two gaps worth separating. Baseline to corrected is what pruning costs. Corrected to released is what the deletion costs on top.

Benchmark	Baseline	Released	Corrected	Pruning cost	Deletion cost
TextVQA	86.2	23.3	80.5	5.7	57.2

Benchmark	Baseline	Released	Corrected	Pruning cost	Deletion cost
DocVQA (ANLS)	94.7	19.2	76.8	17.9	57.6
ChartQA	76.8	29.0	45.4	31.4	16.4
AI2D (control)	86.4	79.6	79.2	7.2	0.4
GQA	60.9	55.9	58.8	2.1	2.9
POPE (accuracy)	87.6	86.2	86.3	1.3	0.1
MME (perception)	1674.5	1658.7	1651.2	23.3	7.5
MMBench-EN	83.7	79.3	79.0	4.7	0.3
ScienceQA-Image	88.1	85.1	85.0	3.1	0.1

Table 5. Qwen2.5-VL-7B. MME is scored out of roughly 2000 rather than 100, so its last two figures are 1.4 and 0.4 per cent of its own baseline. Where a deletion cost would be negative, meaning the released arm scored a shade higher, we give the magnitude.

ChartQA reverses the pattern. There the deletion costs 16.4 points and the pruning costs 31.4, so the discrepancy is not the dominant problem on that benchmark. An earlier version of this analysis treated the deletion as the main cause throughout, which these figures do not support.

6.1 What the standard suite reports

The paper reports six benchmarks. We ran five of them: GQA, MME, MMBench, POPE and ScienceQA. MM-Vet is out because a language model judges it, which adds a second source of variance to a comparison meant to isolate one line of code.

Averaged across those five, the released arm keeps 96.1 per cent of baseline and the corrected arm 96.9. A gap of 0.8 points. On the three reading tasks the same two arms give 28.4 and 77.9 per cent. The suite the method was validated on cannot see the difference.

6.2 The same blind spot without a discrepancy

We then pruned InternVL2-2B to 12.5 per cent with no deletion at all, on an implementation where we found no comparable discrepancy.

Benchmark	Baseline	Pruned to 12.5 %	Retained
TextVQA	0.716	0.468	65.4 %
DocVQA (ANLS)	0.835	0.303	36.3 %
ChartQA	0.586	0.398	67.9 %
AI2D (control)	0.746	0.728	97.6 %
GQA	0.592	0.546	92.2 %
POPE (accuracy)	0.888	0.848	95.5 %
MMBench-EN	83.4	81.0	97.1 %

Table 6. InternVL2-2B, pruning only.

The three reading tasks average 56.5 per cent of baseline. The three from the paper's suite that we ran here, GQA, POPE and MMBench, average 94.9. AI2D is at 97.6 but it is our control, not part of that suite. In points lost: 43.5 against 5.1, about 8.6 to one. No discrepancy is present, the pruning behaves as designed, and the standard benchmarks still report 95 per cent while roughly half the reading ability has gone.

This arm is not a full reproduction of BTP. Our selector has the diversity component but not the attention one, because InternLM2's FlashAttention-2 path does not expose attention scores, and we scaled the pruning layers across from Qwen rather than calibrating them. In one probe on one image, attention picks landed on text 36.0 per cent of the time against a 28.8 per cent share, so attention does seem to favour text. One image cannot tell us by how much, which makes these numbers a likely upper bound on what BTP itself would cost.

7. Interpretation, limitations and next steps

Three findings, at three levels. They stand or fall separately and should not be quoted as one result.

Level	Finding	Scope
Implementation	The released BTP code deletes all remaining image tokens just below the threshold we measured for Qwen2.5-VL, while the paper's Appendix 7.3 asks for retention on that model.	The code at commit 9682db0. Says nothing about author intent, and nothing about Qwen2.5-VL as a model.
Model	Visual token access has a measurable depth threshold, abrupt in both models tested, at 80.3 per cent of decoder depth in Qwen2.5-VL-7B and 74.8 per cent in InternVL2-2B.	Two models, measured on TextVQA. Task-specific and not yet shown to hold for other tasks.
Evaluation	The benchmark suite did not reveal the degradation in text reading, with the discrepancy present or absent.	Five of the paper's six benchmarks, on two models. The one finding that does not depend on the other two.

Table 7. The three findings kept separate.

The evaluation finding generalises furthest and does not depend on the implementation one. On Qwen there is a discrepancy the benchmarks do not reveal. On InternVL there is no such discrepancy, and the benchmarks still do not reveal what pruning costs.

The model-level result is the one we would build on. BTP does calibrate. Section 4.3 of the paper selects the three pruning layers using a fixed set of 64 samples, by locating where image-token representations shift. What we measure is a different quantity: the depth at which removing the remaining tokens entirely becomes safe. That depth does not appear to be calibrated anywhere, and in the released Qwen configuration it is a fixed value. Our two measurements differ by more than five percentage points of decoder depth, which is enough for such a value to fall on the wrong side of the threshold in one architecture while being harmless in another.

The evaluation-level result is the one we would warn about. Both models lost far more on reading tasks than the standard suite reported, in one case with a discrepancy present and in the other without. A suite that contains no reading task cannot distinguish a method that preserves reading from one that destroys it.

7.1 Limitations

- Two architectures are two architectures. The shapes matched, which is not a general law.
- The thresholds were measured on TextVQA. Given ChartQA's behaviour in Table 5, we would not assume the threshold lies in the same place for that task.
- The InternVL implementation is ours, with the selector only partly built and the pruning layers transplanted from Qwen rather than calibrated.
- The OCR oracle ran at one budget. An unexplained slowdown ended the intended sweep, which was not resumed.

- Everything here uses 500 sample subsets, matched across arms, so the comparisons hold but the absolute numbers are not the published full-dataset ones.
- **Two of our own claims are withdrawn.** Phase 2 blamed the collapse on text having no redundancy; our own similarity measurement says the opposite, 0.379 against 0.438. And an early result where text coverage seemed to predict correct answers disappeared at a larger sample. An earlier draft of this report also concluded the effect was Qwen-specific, on one measurement in InternVL, and the sweep showed that was wrong.

7.2 Recommendations

Measure the depth rather than selecting it. The sweep is inexpensive, the tool is written and validated on one model, and it requires no training and no calibration set. Approximately seven evaluations establish the value.

Include at least one reading benchmark in the evaluation. Not because reading matters more than reasoning, but because it degrades first when visual evidence is removed, which makes it the more informative early indicator.

Next: thresholds on DocVQA and ChartQA, taking ChartQA first since pruning already costs more there than the deletion does, then a third architecture, then the manuscript.

7.3 Reproducibility

Artefact	Location
Run-by-run record	phase3-mechanism/RESULTS_LOG.md, observed output marked VERIFIED, untested statements HYPOTHESIS
Scripts and jobs	phase3-mechanism/, job scripts e9 to e13 and their submit chains
Threshold tool	phase3-mechanism/measure_visual_depth.py, runs on any Hugging Face VLM
Figures and this file	report/make_depth_figures.py, make_workflow_figure.py and build_report.js, so every number comes out of a script
Repository	github.com/milab-iitj-dev/btp-reproduction, commit 9214e75
Jobs	411681, 412401, 414662, 416734-5, 416896-904, 417968, 418974-419012, 421205-421237, 423959
The paper	Balanced Token Pruning, NeurIPS 2025, arXiv:2505.22038. Code read at commit 9682db0.

Table 8. Where each part of the work can be checked.